

PestLLM

Codebase Overview

PestLLM — Codebase Overview

A technical walkthrough of the whole project: what it is, how the pieces fit together, and where to look when you need to change something. Written to be read top-to-bottom by someone new to the repo. For setup/deploy steps see the root README.md.

1. What this product is

PestLLM is a **WhatsApp-first AI agent for a pest-control business**. The customer-facing brand is *GreenShield Pest Control*; the assistant introduces itself as "Ajax 1.0". A customer messages the company's WhatsApp number and an LLM (Claude, or any Anthropic-compatible model) runs the entire conversation:

- quotes prices,
- identifies pests from a photo,
- books / reschedules / cancels appointments,
- handles Annual Maintenance Contracts (AMC — renewals & new subscriptions),
- escalates to a human when something is unsafe or out of scope.

Behind that chat sit two web apps and several background jobs:

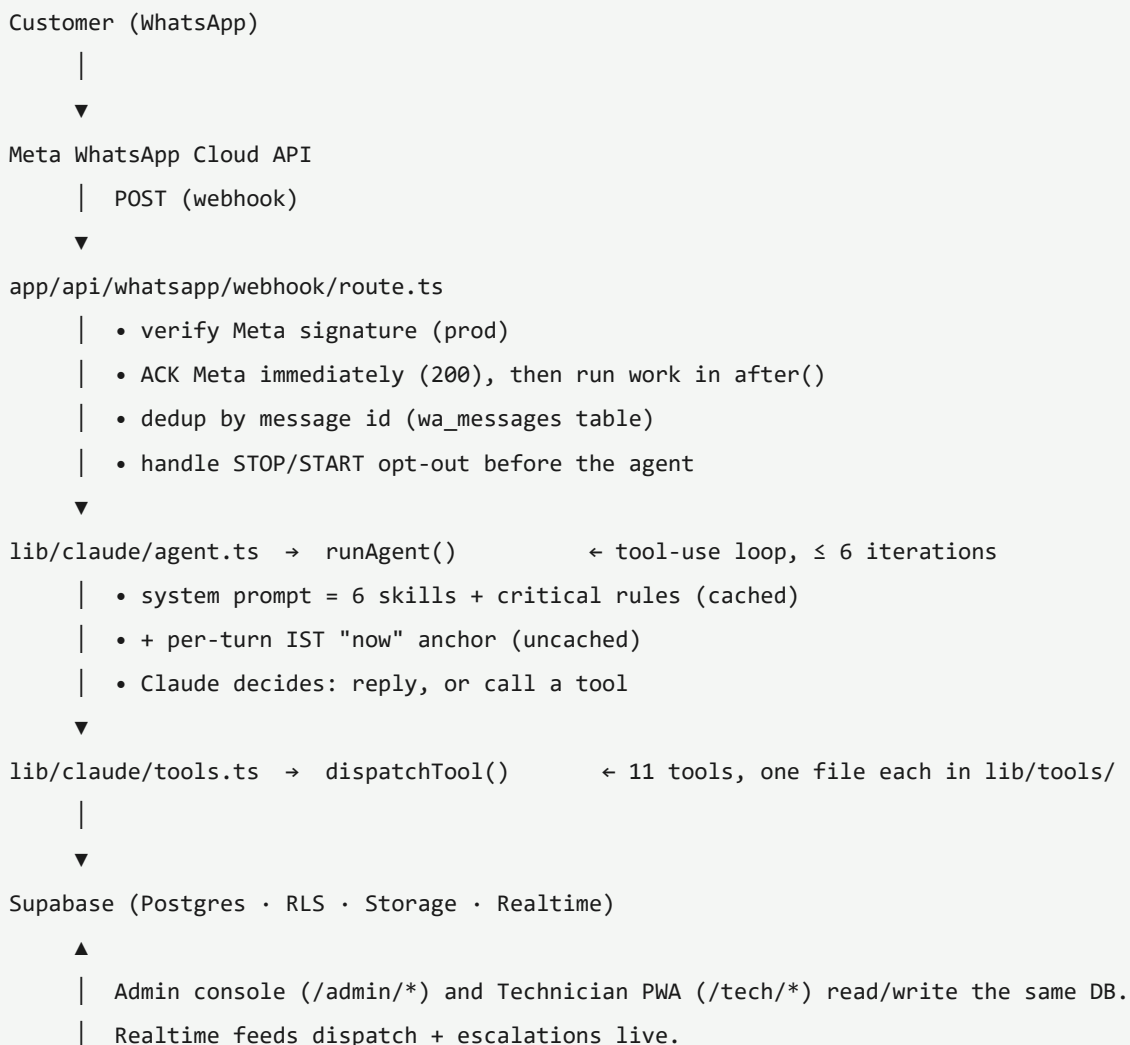
Who	Surface	Purpose
Customer	WhatsApp + <code>/track/[token]</code> , <code>/privacy</code>	The conversation; a live technician-tracking page; a privacy notice.
Staff (admin)	<code>/admin/*</code>	Dashboard, appointments, conversations, dispatch, escalation triage, pricing editor, user management, KPI analytics.
Technician	<code>/tech/*</code>	Installable mobile PWA: today's jobs, job detail (photos, complete), live GPS sharing, escalations, profile.
System	<code>/api/cron/*</code>	Scheduled outreach: reminders, nudges, AMC lifecycle, data-retention purge.

The app targets the **India market**: all times render in `Asia/Kolkata` (IST, UTC+5:30, no DST) and it ships DPDP-Act-aware privacy, opt-out handling, and a retention cron.

2. Tech stack

- **Next.js 15** (App Router, Server Actions, route handlers) · **React 19** · **TypeScript** · **Tailwind**
 - **Anthropic SDK** (`@anthropic-ai/sdk`) — tool-use agent; model is env-driven (`ANTHROPIC_MODEL` , default `claude-sonnet-4-6`). A separate faster/vision model handles pest photos.
 - **Supabase** — Postgres, Auth, Storage (`job-photos` bucket), Realtime. **Row-Level Security (RLS) is the real security boundary.**
 - **WhatsApp Cloud API** (Meta Graph `v23.0`) — inbound webhook + outbound text & template messages.
 - **Vercel** — hosting + cron scheduling (vercel.json).
 - Optional: **LiteLLM** proxy (litellm-config.yaml) to run against a local Ollama model in dev without an Anthropic key.
-

3. The big picture (request flow)



Two concepts drive the agent's behaviour, and they are deliberately separate:

- **Skills** = *instructions* (skills/*.md). Markdown files

concatenated into the cached system prompt. **No code runs.** They shape how the agent talks and when it should reach for a tool.

- **Tools** = *code Claude can call* (lib/tools/*.ts). Each has a

JSON schema in lib/claude/tools.ts and a dispatcher entry. This is where anything with a side effect (DB writes, sends) lives.

4. The agent loop in detail — lib/claude/agent.ts

`runAgent({ history, userText, mediaUrls, ctx })` is the heart of the product.

1. **Assemble the prompt.** The system message has two blocks:

- the **cached** block: critical rules + the 6 skills (stable, so Anthropic

prompt-caching keeps it cheap across turns);

- an **uncached** "now" block giving the current IST date/time so the model can

resolve "today"/"tomorrow" without invalidating the cache. 2. **Build the user turn** from `userText` + any inbound photos (base64 image blocks). Empty messages get a `(empty message)` placeholder. 3. **Loop up to**

`MAX_TOOL_ITERATIONS = 6` : call the model; if it stops with `tool_use` , run every requested tool via `dispatchTool` , append the results, and loop again. Otherwise collect the text and finish. 4. **Persist a trimmed, image-stripped history.** Two important hygiene steps:

- `stripImages` replaces stored base64 photos with a text stub — otherwise

every past photo would be re-sent on every future turn and balloon the `conversations.state_json` row.

- `trimHistory` keeps the last ~30 turns but only ever cuts at a genuine user

turn, so it never strands a `tool_result` without its matching `tool_use` (which the API rejects with a 400).

Anthropic-only features (`thinking` , `cache_control`) are gated behind an `IS_CLAUDE` flag so a compat backend (Kimi/Ollama via LiteLLM) still works.

The 6 skills (skills/)

`persona` · `escalation-and-safety` (kept near the top so safety isn't under-weighted) · `booking-flow` · `pricing-and-quotes` · `pest-identification` · `amc` . Assembled by lib/claude/skills.ts, which also prepends **CRITICAL RULES** (never fabricate prices/slots/codes; copy `slot_start` character-for-character; never claim a booking is confirmed before `create_appointment` returns a code; escalate on any bite/sting/complaint/safety).

The 11 tools (lib/tools/)

Tool	What it does
<code>check_availability</code>	Open slots in a date range for a service tier. Must be called before proposing slots.
<code>create_appointment</code>	Books a slot; returns a 6-char confirmation code. Auto-assigns a technician.
<code>reschedule_appointment</code>	Moves a booking to a new slot (needs the confirmation code).
<code>cancel_appointment</code>	Cancels a booking (needs the confirmation code).
<code>get_pricing_quote</code>	Price range from the <code>pricing</code> rate card; flags if an on-site inspection is required.
<code>identify_pest</code>	Vision — identifies the pest from the customer's inbound photo (or a text description). Uses the faster/vision model.
<code>escalate_to_human</code>	Flags the conversation for staff (safety, complaints, specialist work).
<code>lookup_customer</code>	Prior visits / stored details by WhatsApp phone (already in context).
<code>lookup_amc_status</code>	Whether this customer has an AMC on file, with plan details.
<code>request_amc_renewal</code>	Customer confirmed renewal → admin escalation + <code>pending_renewal</code> .
<code>request_amc_subscription</code>	New customer wants an annual plan → admin escalation to quote & collect payment.

Context injection is a deliberate security pattern. `dispatchTool` injects `customer_phone` from the verified WhatsApp sender (`ToolContext`) rather than letting the model pass it — so the agent can't act on behalf of another customer. Likewise `identify_pest` pulls the photo from `ctx.mediaUrls` , never from model-emitted image data (a data-URL round-tripped through tool output would cost thousands of tokens and be spoofable).

5. WhatsApp integration — `lib/whatsapp/` + the webhook

Inbound (`lib/whatsapp/inbound.ts`):

- `handleVerification` answers Meta's GET webhook-verification handshake.
- `verifyMetaSignature` validates `X-Hub-Signature-256` (HMAC, timing-safe) — enforced in production.
- `parseInboundMessages` walks the **full** batch (`entry[]` → `changes[]` → `messages[]`); Meta can pack several messages into one POST, so handling only the first would silently drop the rest.
- Media (image/video/audio/document/sticker) is resolved from a Graph media id to a **base64 data: URL** server-side (the download needs the same bearer token), so downstream consumers (Claude vision) never re-authenticate.

The webhook (`app/api/whatsapp/webhook/route.ts`):

- **ACKs Meta immediately** with a 200 and does the real work in Next's

`after()` — so multi-step tool chains never blow Meta's response window.

- **Idempotency:** claims the Meta message id in the `wa_messages` table; a

unique-violation (`23505`) means a redelivery already ran it → skip.

- **Opt-out first:** STOP/START is detected and applied *before* the agent runs.
- Processes a customer's messages **sequentially** so rapid double-texts don't

race on the same conversation history.

- On agent crash, it messages the customer a graceful failure line **and** files

an escalation so a human follows up (never goes silent).

Outbound (lib/whatsapp/outbound.ts) — the proactive-messaging safety layer. Every send must declare a **kind**:

- **transactional** (booking confirmations, reminders, en-route) — always sent;
- **promotional** (nudges, AMC upsell) — **suppressed for opted-out customers**.

`sendWhatsAppToCustomer` / `sendTemplateToCustomer` force the caller to pass the kind so opt-out filtering can't be bypassed. Sends outside Meta's 24-hour window must use pre-approved **templates** (lib/whatsapp/templates.ts).

6. Scheduled jobs — `app/api/cron/*` (vercel.json)

Route	Schedule	Purpose
<code>/api/cron/reminders</code>	hourly	24h-out appointment reminders (once per booking, tracked via <code>reminder_sent_at</code>).
<code>/api/cron/nudges</code>	every 10 min	One re-engagement nudge for idle threads (free-form; always inside the 24h window).
<code>/api/cron/retention</code>	daily 18:00	DPDP: purge chat logs older than <code>RETENTION_CONV_MONTHS</code> (default 6) + prune webhook-dedup rows.
<code>/api/cron/amc</code>	daily 01:00	AMC renewal reminders, follow-ups, and upsell pitches.

All cron routes require `Authorization: Bearer $CRON_SECRET` and **fail closed** in production — if `CRON_SECRET` is unset they return 503 rather than being publicly triggerable. Vercel attaches the header automatically.

7. Data model — `supabase/migrations/`

Apply migrations **in filename order**. Core tables (from 0001_init.sql onward):

- **customers** — phone (unique), name, address, notes; later `opted_out`.
- **conversations** — one row per customer; `state_json` holds the trimmed

agent history; `last_message_at` drives nudges.

- **appointments** — `confirmation_code` (unique), `pest_type`, `service_tier`

(`standard|plus|specialist`), `slot_start/end`, status (`booked|cancelled|completed`), `price_quoted`, plus later `assigned_technician_id`, `reminder_sent_at`.

- **pricing** — rate card keyed by (`pest_type`, `service_tier`): `base_price`,

`per_sqft`, `requires_inspection`. Seeded with ~11 pest/tier rows.

- `escalations` — `customer_id`, `summary`, `urgency`, `resolved`.

Layered on by later migrations:

- `profiles` + RLS (`0002_profiles_and_rols`) — role `admin | technician` ; the `is_admin()` helper backs most policies.
- `nudge` column (`0002_nudge`).
- `technician phase` (`0003`) — photos + the `job-photos` Storage bucket.
- `GPS tracking` (`0004`) — `technician_positions` , tracking tokens, trip state.
- `RLS hardening + tech column guard` (`0005` , `0006`).
- `opt-out` (`0007`).
- `AMC` (`0008_amc`) — annual maintenance contracts (`pest_type`, `annual_price`, `renews_at`, `status`).
- `deployment tier` (`0009`) — singleton `deployment_settings` row (see §9).
- `db refinement` (`0010`) — indexes + integrity constraints, incl. the DB-level double-booking guard.

Capacity model: one booking per slot globally (three fixed windows/day),

enforced by a unique index — regardless of how many technicians exist. Raising

throughput means moving to per-technician slot capacity.

8. Web surfaces

Auth & routing

Supabase Auth + a `profiles.role` . `middleware.ts` refreshes the session cookie and gates `/admin` vs `/tech` by role (and bounces logged-in users off `/login`). It's a UX/redirect layer only — **RLS is the real boundary**. New sign-ups default to `technician` ; promote your first admin with SQL.

Admin console — `app/(admin)/admin/*`

Overview dashboard, appointments, conversations viewer, **dispatch** (assign technicians + live positions via Realtime), **escalation** triage, **pricing** editor, **user/role** management, and a tier-gated **KPI** dashboard.

Technician PWA — `app/(tech)/tech/*`

Installable mobile app (`app/manifest.ts`): today's assigned jobs, job detail (notes, before/after photos to `job-photos` , mark complete), live **en-route / arrived** GPS sharing, escalations, profile.

Public

- `/track/[token]` — token-gated live-tracking page. `lib/tracking.ts`

only serves a position that **belongs to this appointment** and is **< 15 min old** — defense-in-depth so a stale link can't read the tech's location on a later, different customer's job. Links out to Google Maps rather than embedding.

- `/privacy` — DPDP Act notice with configurable company/DPO details (`lib/legal.ts`).

9. KPI analytics & deployment tiers

Tiers (lib/tier.ts, `deployment_settings` singleton): each Supabase project is one client deployment. `tier2` is the default; flipping to `tier3` exposes the KPI dashboard in the admin console. Defaults **fail safe** to `tier2` on any error, so tier-3 features stay hidden rather than leak.

KPI queries (lib/kpi/queries.ts) compute funnel, financial, operational, technician, and AMC metrics over an IST date range (`week|month|quarter` , with previous-period comparison). Since the booking tool doesn't record `price_quoted` today, revenue is **estimated** from per-tier average base prices (`computeTierAverages` / `valueOf`) so figures aren't perpetually zero.

10. Supporting libraries — lib/

- `lib/supabase/{server,client,browser,types}.ts` — three clients for three

contexts: `server` = anon SSR client (respects RLS), `client` = **service-role** (webhook/cron/server, bypasses RLS), `browser` = client-side. `types.ts` holds shared DB types (`ServiceTier` , `DeploymentTier` , etc.).

- `lib/time.ts` — `BUSINESS_TZ = Asia/Kolkata` , `parseBusinessTime` (IST → UTC Date).
 - `lib/geo.ts` — geocoding via OpenStreetMap **Nominatim** (no API key); ETAs are straight-line **haversine** estimates.
 - `lib/auto-assign.ts` — `pickTechnician()` used at booking time.
 - `lib/tracking.ts` — the position-serving gate described in §8.
 - `lib/tier.ts` , `lib/legal.ts` , `lib/brand.ts` , `lib/utils.ts` — deployment tier, privacy/legal text, brand strings, misc helpers.
-

11. Repo map (where to look)

```
app/api/whatsapp/webhook/route.ts    Meta inbound (GET verify + POST); opt-out; hands off to runAgent
app/api/cron/{reminders,nudges,retention,amc}/route.ts    Scheduled outreach
app/api/track/[token]/route.ts      Public position read for the tracking page
app/(admin)/admin/*                  Staff console
app/(tech)/tech/*                    Technician PWA
app/track/[token]/ · app/privacy/    Public customer pages
lib/claude/{agent,skills,tools,client}.ts    Tool-use loop, prompt assembly, tool schemas
lib/tools/*.ts                        One file per tool (side effects live here)
lib/whatsapp/{client,inbound,outbound,opt-out,templates}.ts
lib/supabase/{server,client,browser,types}.ts
lib/kpi/queries.ts                   Admin analytics
lib/{time,geo,tracking,legal,tier,brand,auto-assign,utils}.ts
skills/*.md                          Persona + 5 topical skills (system prompt)
supabase/migrations/*.sql            Schema evolution (apply in order)
scripts/*                            Smoke tests, seeding, tier tooling, deploy
docs/                                Client brief, onboarding, this overview
marketing/                           Lead-gen reels + explainer (not part of the app runtime)
```

12. Known gaps & non-obvious constraints

- **Voice notes aren't understood** — inbound audio is dropped before the agent runs; the customer gets a generic reply.
- **Global (not per-tech) slot capacity** — one booking per slot regardless of technician count (see §7).
- **Revenue is estimated** — `price_quoted` isn't written at booking time, so KPIs approximate from tier averages (§9).
- **Two migrations share the `0002_` prefix** (`0002_nudge.sql` , `0002_profiles_and_rls.sql`) — apply both.
- **Straight-line ETAs / no embedded map** — Nominatim + haversine; tracking links out to Google Maps.
- **Out of scope**: payments/deposits and a voice channel.
- On this dev machine, npm/dev need `NODE_OPTIONS="--use-system-ca"` for TLS.